



MediCare: Clinic Management & Appointment System

1. Project Metadata & Identification

Field	Value / Details
Project Title	MediCare — Clinic Management & AI-Assisted Appointment System
DEPI Track	Full Stack .NET Web Developer
Core Architecture	ASP.NET Core MVC (3-Layer N-Tier: MediCare.Web, MediCare.Services, MediCare.Data)
Group / Wave Code	HRV | #13 | ONL5_SWD5_S3 | DEPI5
Supervising Instructor	Eng : Margret Youssef
GitHub Repository URL	https://github.com/MO7AMED-EL3NTABLY/Graduation-project-Full-stack.net-DEPI-
Target Submission Window	October 2026 – December 2026

2. Project Proposal

2.1 Motivation & Problem Description

- **1. Booking Friction & Multi-Location Confusion:** Patients call clinics to find open slots, cannot see when a doctor is available across multiple workplaces, do not know how many patients are queued ahead of them, and manual schedules cause double bookings.

- **2. Treatment Non-Adherence:** Patients frequently forget medication times, requested radiology scans, and follow-up visits, resulting in missed doses and delayed recovery.

- **3. Difficulty Reading Prescriptions & Lab Reports:** Patients struggle to recall what each prescribed drug is for or its side effects, and lab/radiology reports contain dense medical terminology and reference ranges.

- **4. Doctor Overload & Memory Gaps:** Doctors examine dozens of patients daily and cannot instantly recall every previous visit or prescription, while patients cannot easily verify a doctor's portfolio, hospital experience, or surgical background.

- **5. Disconnected Pharmacies & Radiology Centers:** No direct digital link exists between a doctor and contracted pharmacies or scan centers, causing patients to miss clinic-negotiated discounts.

- **6. Clinic Administration Overhead:** Administrators manage doctor verifications, medical specializations, and partner clinics manually without visual analytics on monthly bookings or revenue.

2.2 Proposed MVC Solution & Methodology

- **3-Layer N-Tier MVC Architecture:** Razor views, Bootstrap 5 layouts, FullCalendar.js, and Chart.js reside in `MediCare.Web`; scheduling rules, conflict checks, and external integrations reside in `MediCare.Services`; EF Core repositories (`IDoctorRepository`, `IPatientRepository`, `IAppointmentRepository`, `IMedicalRecordRepository`) and `UnitOfWork` reside in `MediCare.Data`.

- **Multi-Role ASP.NET Identity Security:** Role-based authentication (`Patient`, `Doctor`, `Admin`). Newly registered doctors remain inactive until an Admin reviews their

license and graduation documents. Admins view aggregate clinic statistics without accessing private patient clinical notes.

- **Location-Aware FullCalendar Booking & Queue Engine:** Doctors define working days and hours per clinic/hospital location. Slots are rendered on FullCalendar.js, database transactions prevent double booking, and patients see their queue number.

- **Digital Prescriptions & Automated Reminders:** Prescriptions store medication dosage, daily frequency, duration, and required scans, with a 'Mark as taken' patient tracker. Background jobs dispatch reminders 24 hours and 2 hours prior to visits via SignalR, SendGrid (email), and Twilio (SMS).

- **Safe Back-End AI Assistant (Groq LLM & PdfPig):** Invoked strictly from `MediCare.Services` (never exposing API keys to the browser): (1) suggests a medical specialty from patient symptoms and lists matching doctors; (2) explains prescribed drug purposes/side effects with a mandatory 'consult your doctor' notice; (3) extracts text from uploaded PDF lab/radiology reports via PdfPig and summarizes values next to normal ranges.
- **Pre-Visit Doctor Summary & Partner Discounts:** Doctors receive a pre-visit summary of the patient's last visit, prescription, and current complaint, and link contracted pharmacies/radiology centers so patients can claim discount codes.

3. Project Plan & 4-Week Execution Timeline

Sprint	Timeline	Core Technical Tasks (DEPI MVC Standard)	Deliverables
Week 1: Setup & User Management	Week 1	Set up N-Tier MVC solution (MediCare.Web, MediCare.Services, MediCare.Data); design EF Core tables (Doctors, Patients, Appointments, Specializations, WorkingHours/Locations, MedicalRecords, Partners); implement Generic & Specific Repositories + Unit of Work; configure ASP.NET Identity for Patient, Doctor, Admin.	Complete 3-layer N-Tier MVC setup, EF Core schema & repositories, multi-role authentication, and Doctor listing page with search/filter by specialization, location, and availability.

Sprint	Timeline	Core Technical Tasks (DEPI MVC Standard)	Deliverables
Week 2: Appointment System & Doctor Dashboard	Week 2	Integrate FullCalendar.js for location-aware slot booking, cancellation, rescheduling, queue display, and conflict detection; build Doctor Dashboard (daily/weekly calendar & list views, locations & working hours, vacation days, portfolio, accept/decline requests); wire SignalR for instant notifications.	Interactive FullCalendar booking system with conflict handling, Doctor Dashboard, and real-time SignalR alerts.
Week 3: Medical Records, Prescriptions, AI & Alerts	Week 3	Build Medical Records module (visit diagnosis, symptoms, lab/X-ray file uploads, pre-visit summary); build Printable Digital Prescription system with 'Mark as taken' tracker; integrate Twilio SMS and SendGrid email reminders (24h & 2h before); integrate server-side Groq AI & PdfPig helper.	Medical records & printable prescriptions, Twilio/SendGrid reminders, safe AI symptom/prescription/report explainer, and Patient profile portal.
Week 4: Admin Dashboard, Analytics & Deployment	Week 4	Build Admin Dashboard (review doctor documents & approve registrations, manage patients, specializations, and pharmacy/radiology partners); build Chart.js analytics with PDF/Excel export; polish Bootstrap 5 UI with Dark/Light mode; deploy to Microsoft Azure with GitHub Actions CI/CD.	Admin dashboard with Chart.js reports & PDF/Excel export, responsive Dark/Light UI, live Azure deployment, and complete project documentation.

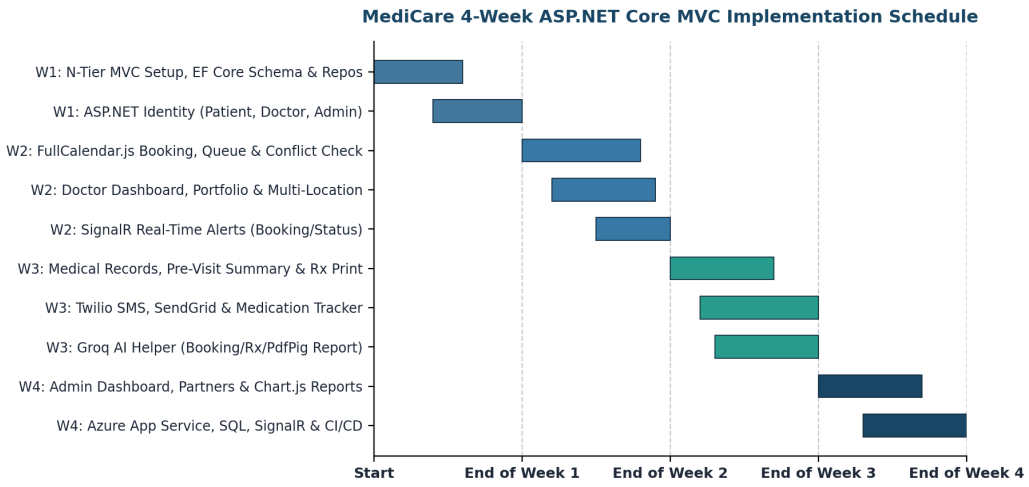


Figure 1: MediCare 4-Week ASP.NET Core MVC Sprint Plan and Milestone Schedule.

4. Task Assignment & Team Roles

Member	Role	Main responsibilities
Member 1 – [omar hussien]	Team Lead, Appointments	Doctor listing with search and filters, working hours per location, vacation days, booking with FullCalendar, conflict detection, cancel and reschedule, queue visibility, doctor dashboard
Member 2 – [heba hossam]	Core	Solution structure (Web, Services, Data), generic repositories, Unit of Work, Identity with three roles and registration flows, doctor approval, admin panel and clinic settings, Azure deployment, CI/CD
Member 3 – [mohammed elsaeid]	Clinical records and Partners	Medical records and uploads, prescriptions with print format, treatment schedule, prescription QR code and verification page, pharmacies and radiology centers with discounts, patient profile
Member 4 – [youssef saad]	AI Assistant	Groq integration from the back end, booking agent with function calling, prescription explainer, report explainer (PDF text extraction), AI safeguards and logging, pre-visit summary
Member 5 – [reham ahmed]	Notifications, UI and Analytics	SignalR notifications, reminder scheduling, SendGrid and Twilio, notification preferences, Bootstrap 5 UI and dark mode, Chart.js analytics, PDF/Excel export

Team practices

- GitHub feature-branch workflow: main (stable), develop (integration), feature/<module> per task.
- Every merge goes through a pull request reviewed by at least one other member.

Short weekly meeting to review progress and blockers, and a task board (GitHub Projects) with To Do, In Progress, Review and Done.

5. Risk Assessment & Mitigation Plan

Risk Event	Severity	Likelihood	Practical Mitigation Strategy
Concurrent double-booking of the same doctor and location slot	High	Medium	Enforce database-level unique filtered index on (DoctorId, WorkingHourId, AppointmentDate, StartTime) where Status != 'Cancelled' inside a Unit of Work transaction.
Groq API, Twilio, or SendGrid rate limits during testing/defense	Medium	High	Wrap external calls behind service interfaces in MediCare.Services with graceful fallback responses and keep API keys strictly in server User Secrets / Azure Environment Variables.
Patients misinterpreting AI prescription or lab report summaries	High	Low	Restrict prompts strictly to non-diagnostic explanations, strip personal identifiers before calling Groq, and append a fixed 'Consult your doctor' disclaimer in C# code.
SQL Server cascade path conflicts on Appointments foreign keys	Medium	High	Configure OnModelCreating with DeleteBehavior.Restrict on DoctorId and PatientId relationships.

6. Key Performance Indicators (KPIs)

- **Booking Conflict Rate (Target: 0%):** Zero overlapping active reservations per doctor location under simultaneous requests.
- **MVC View & Calendar Feed Latency (Target: < 400ms):** Doctor filtering and FullCalendar JSON slot feeds respond in under 400ms.
- **Real-Time Notification Delivery (Target: < 2 seconds):** SignalR alerts reach the doctor or patient browser within 2 seconds of a booking or status update.
- **Service Layer Unit Test Pass Rate (Target: >= 80% coverage):** Core booking conflict rules, prescription trackers, and repository queries validated via automated xUnit tests.